



THE UNIVERSITY OF
MELBOURNE

SWEN90010 - High Integrity Systems Engineering

Ada

Toby Murray



toby.murray@unimelb.edu.au



DMD 8.17 (Level 8, Doug McDonnell Bldg)



<http://people.eng.unimelb.edu.au/tobym>



@tobycmurray



THE UNIVERSITY OF
MELBOURNE

INTRODUCING *Ada*

Augusta Ada King (née Byron)



THE UNIVERSITY OF
MELBOURNE

aka **Ada**, Countess of **Lovelace**

10 Dec 1815 - 27 Nov 1852

16 Jan 1816: Lord Byron departs

Studied math at early age,
tutored by De Morgan

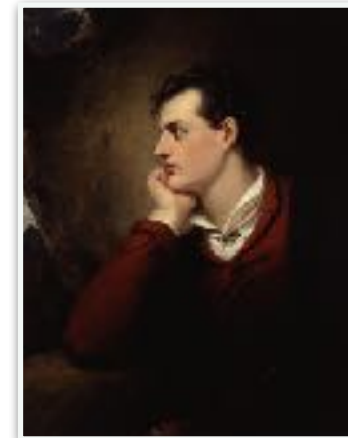
June 1833 (age 17)

1842-43: Analytical Engine
translation plus notes

$$B_m^+ = 1 - \sum_{k=0}^{m-1} \binom{m}{k} \frac{B_k^+}{m-k+1}$$

Bernoulli Numbers

**Analytical Engine as
Logical Symbol
Manipulation Engine**



Ada

mid 1970s: US DoD has too many languages for myriad
embedded devices;

want: safe, modular, cross platform, high level

1977: solicit proposals for new language

1979: Honeywell's proposal chosen; christened **Ada**

10 Dec 1980: MIL-STD-**1815** approved

1983: ANSI/MIL-STD-1815A ("Ada 83")

1995: ISO-8652:1995 ("Ada 95")

USAF funds GNAT Compiler, now in GCC

2012: ISO/IEC 8652:2012 ("Ada 2012")

Strong, Static Typing



THE UNIVERSITY OF
MELBOURNE

```
procedure StrongTyping is
  I : Integer := 0;
  J : Float := 0.0;
begin
  I := J;
end StrongTyping;
```

```
int main(void){
  int i = 0;
  float f = 0.0;
  i = f;
  return 0;
}
```

```
$ gnatmake StrongTyping.adb
gcc -c strongtyping.adb
strongtyping.adb:7:09: expected type "Standard.Integer"
strongtyping.adb:7:09: found type "Standard.Float"
gnatmake: "strongtyping.adb" compilation error
$
```

Prevents e.g.
loss of precision

```
$ gcc strong_typing.c -o strong_typing
$
```

C: weak, static typing

Module System



THE UNIVERSITY OF
MELBOURNE

```
package ModuleOne is
  procedure DoNothing;
end ModuleOne;
```

```
package ModuleTwo is
  procedure DoNothing;
end ModuleTwo;
```

```
with ModuleTwo;

package body ModuleOne is
  procedure DoNothing is
  begin
    ModuleTwo.DoNothing;
  end DoNothing;
end ModuleOne;
```

```
package body ModuleTwo is
  procedure DoNothing is
  begin
    null;
  end DoNothing;
end ModuleTwo;
```

```
$ gnatmake ModuleOne.adb
gcc -c moduleone.adb
gcc -c moduletwo.adb
$
```

Automatic dependency resolution
Information Hiding, Encapsulation

Portable

“For a signed integer type, the exception **Constraint_Error** is raised by the execution of an operation that cannot deliver the correct result because it is outside the base range of the type.”

```
procedure Overflow is
  I : Integer := 0;
begin
  loop
    I := I + 1;
  end loop;
end Overflow;
```

```
$ gnatmake Overflow.adb
gcc -c overflow.adb
gnatbind -x overflow.ali
gnatlink overflow.ali
$ ./overflow

raised CONSTRAINT_ERROR : overflow.adb:5 overflow check failed
$
```

Portable (c.f. C)

(Cost of Runtime Checking)

```
int main(void){  
    int i = 0;  
    while(i >= 0){  
        i++;  
    }  
    return 0;  
}
```

```
$ gcc overflow.c -o overflow  
$ ./overflow  
$
```

```
$ time ./overflow  
  
raised CONSTRAINT_ERROR  
real 0m4.775s  
user 0m4.770s  
sys 0m0.003s
```

```
$ time ./overflow_c  
  
real 0m4.072s  
user 0m4.066s  
sys 0m0.004s
```

Is this any safer?

But stronger static checking
can **improve** performance too.
(see later)

Readability



THE UNIVERSITY OF
MELBOURNE

Purposeful Verbosity

```
procedure Overflow is
  I : Integer := 0;
begin
  loop
    I := I + 1;
  end loop;
end Overflow;
```

```
int main(void){
  int i = 0;
  while(i >= 0)
    i++;
  return 0;
}
```

```
package ModuleOne is
  procedure DoNothing;
end ModuleOne;
```

```
$ time ./overflow

raised CONSTRAINT_ERROR
real 0m4.775s
user 0m4.770s
sys 0m0.003s
```

Intended Redundancy

Unambiguous Semantics

Ada Prefers Readability

Make **reading** programs **easier** than **writing** them.

Why?

Code may be read > 10 times,
but written only once.

“First, optimise the task that
contributes most to overall cost.”

This includes **programs**
that read your code

JavaScript:

```
>>> false == 'false';  
false  
>>> false == '0';  
true  
>>> '' == '0';  
false  
>>> 0 == '';  
true  
>>> 0 == '0';  
true
```

c.f. Ruby, Javascript, Perl which do the opposite

Runtime Checks

Signed overflow

Array index out-of-bounds

Accessing unallocated memory

Range errors (see later) etc.

Turns **heisenbugs** into **crashes**

Implications for safety of real-time embedded,
and critical systems?

Application Areas

Mostly real-time embedded

Avionics

Railway

Defence

Space

Robotics

Crypto(graphy)

Muen verified Separation Kernel

Lots of these
with formal verification
via SPARK

MISRA C

MISRA C: “safe” C subset used in automotive
(not all rules enforceable)

Ada is already much “safer” than C; better encapsulation

SPARK: an “even safer” subset of Ada, with
same runtime semantics (future lectures)

Ada: stronger type system, more static guarantees

Ada: more runtime checks
(e.g. array index out of bounds,
dynamic memory access etc.)



ADA: SOME INTERESTING HIGHLIGHTS

Packages



```
package Point is  
  
  type Point is record  
    X : Integer;  
    Y : Integer;  
    Z : Integer;  
  end record;  
  
  procedure PrintPoint(P : Point);  
  
end Point;
```

Spec
the **interface**
says “what”

says “how”
the **implementation**
Body

```
with Ada.Text_IO; use Ada.Text_IO;  
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;  
  
package body Point is  
  procedure PrintPoint(P : in Point) is  
  begin  
    Put("X: "); Put(P.X);  
    Put(" Y: "); Put(P.Y);  
    Put (" Z: "); Put (P.Z);  
    New_Line;  
  end PrintPoint;  
end Point;
```

Packages: **with** and **use**



```
with Ada.Text_IO; use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

package body Point is
  procedure PrintPoint(P : in Point) is
  begin
    Put("X: "); Put(P.X);
    Put(" Y: "); Put(P.Y);
    Put (" Z: "); Put (P.Z);
    New_Line;
  end PrintPoint;
end Point;
```

```
with Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

package body Point is
  procedure PrintPoint(P : in Point) is
  begin
    Ada.Text_IO.Put("X: "); Put(P.X);
    Ada.Text_IO.Put(" Y: "); Put(P.Y);
    Ada.Text_IO.Put (" Z: "); Put (P.Z);
    Ada.Text_IO.New_Line;
  end PrintPoint;
end Point;
```

Types



Type equality is by name

What
errors does
this catch
statically?

```
procedure TypeEquality is
  type FirstType is range 0..10;
  type SecondType is range 0..10;
  I : FirstType := 0;
  J : SecondType := 0;
begin
  I := J;
end TypeEquality;
```

Will this code compile correctly?

```
$ gnatmake typeequality.adb
gcc -c typeequality.adb
typeequality.adb:7:09: expected type "FirstType" defined at line 2
typeequality.adb:7:09: found type "SecondType" defined at line 3
gnatmake: "typeequality.adb" compilation error
```

Type Casts

```
procedure TypeCast is
  type FirstType is range 0..10;
  type SecondType is range 0..10;
  I : FirstType := 0;
  J : SecondType := 0;
begin
  I := FirstType(J);
end TypeCast;
```

This compiles and runs without error.

Type Casts and Errors

```
with Ada.Integer_Text_IO;  
  
procedure TypeCast_Error is  
  type BigType is range 0..20;  
  type LittleType is range 0..10;  
  I : BigType := 0;  
  J : LittleType := 0;  
begin  
  Ada.Integer_Text_IO.Get(Integer(I));  
  J := LittleType(I);  
end TypeCast_Error;
```

Compiles without error.

```
$ ./typecast_error  
10  
$ ./typecast_error  
20  
  
raised CONSTRAINT_ERROR : typecast_error.adb:10 range check failed  
$ ./typecast_error  
30  
  
raised CONSTRAINT_ERROR : typecast_error.adb:9 range check failed
```

Subtypes



```
procedure Subtypes is
  subtype FirstType is Integer range 0..10;
  subtype SecondType is Integer range 0..10;
  I : FirstType := 0;
  J : SecondType := 0;
begin
  I := J;
end Subtypes;
```

This compiles and executes with no errors.

Casting is effectively implicit between subtypes.

Subtypes



THE UNIVERSITY OF
MELBOURNE

```
procedure Subtypes_Error is
  subtype BigType is Integer range 0..20;
  subtype LittleType is Integer range 0..10;
  I : BigType := 20;
  J : LittleType := 0;
begin
  J := I;
end Subtypes_Error;
```

What happens when we run this?

```
$ ./subtypes_error
```

```
raised CONSTRAINT_ERROR : subtypes_error.adb:7 range check failed
```

btw

```
$ gnatmake subtypes_error.adb
gcc -c subtypes_error.adb
subtypes_error.adb:7:09: warning: value not in range of type "LittleType" defined at line 3
subtypes_error.adb:7:09: warning: "Constraint_Error" will be raised at run time
gnatbind -x subtypes_error.ali
gnatlink subtypes_error.ali
$
```


Arrays

```
procedure Array_Examples is
  Int_Index : array(Integer range 5..10) of Character :=
    ('a', 'b', 'c', 'd', 'e', 'f');
  Ch_Index : array(Character) of Character;
begin
  Put(Int_Index(5)); -- accessing using an integer index
  New_Line;

  Ch_Index('a') := 'z'; -- setting using a char index
  Put(Ch_Index('a')); -- accessing using a char index
  New_Line;

  -- setting and getting an array 'slice'
  Int_Index(6 .. 8) := ('X', 'Y', 'Z');

  Put(Int_Index'First); -- array attribute 'First'
  Put(Int_Index'Last); -- array attribute 'Last'
  New_Line;

  -- attributes 'Range' and 'Length'
  for Index in Ch_Index'Range loop
    Ch_Index(Index) := 'a';
  end loop;
  Put(Ch_Index'Length);
  New_Line;
end Array_Examples;
```

Procedures and Functions

```
procedure ProcFunc is
  X : Integer := 0;
  procedure Proc(I : Integer) is
  begin
    X := I;
  end Proc;

  function Func(I : Integer) return Integer is
  begin
    X := I;
    return I;
  end Func;

  Y : Integer := 0;
begin
  Proc(X+1);
  Y := Func(X+2);
  Ada.Integer_Text_IO.Put(Y);
  Ada.Integer_Text_IO.Put(X);
end ProcFunc;
```

Nesting, static
scope

Functions
with side-effects
(disallowed in SPARK)

Parameter Passing

```
with Ada.Text_IO; use Ada.Text_IO;
with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

procedure Calling_Subprograms is
  An_Int : Integer := 50;
  Another_Int : Integer := 60;
begin
  Put(An_Int, 20); -- positional parameter
  New_Line;       -- no parameters
  Put(Width => 20, Item => An_Int); -- named associations
end Calling_Subprograms;
```

names come from names of **formal parameters**
(which is why they must match between
package spec and body)

Parameter Modes

```
package Vector is  
  
  type Vector is record  
    X : Float;  
    Y : Float;  
    Z : Float;  
  end record;  
  
  procedure Init(V: out Vector);  
  procedure Print(V : in Vector);  
  procedure Normalise(V : in out Vector);  
  
end Vector;
```

in: read only

out: write only

in out: read/write

Checked statically by the type system

Allows e.g. all records to be passed by pointer
if the compiler desires.

Note: Strong typing *improving* performance over e.g. C

Implementation Hiding

```
package Vector is

  type Vector is private;

  procedure Set(V: out Vector; X : in Float; Y : in Float; Z : in Float);
  procedure Print(V : in Vector);
  procedure Normalise(V : in out Vector);

private
  type Vector is record
    X : Float;
    Y : Float;
    Z : Float;
  end record;

end Vector;
```

Vector's fields can't be accessed outside the package.

Private



```
package Vector is

  type Vector is private;

  procedure Set(V: out Vector; X : in Float; Y : in Float; Z : in Float);
  procedure Print(V : in Vector);
  procedure Normalise(V : in out Vector);

private
  type Vector is record
    X : Float;
    Y : Float;
    Z : Float;
  end record;

end Vector;
```

Why does the **Vector** record need to be defined in the spec?

Private: A Hint

Why does the **Vector** record need to be defined in the spec?

```
with Vector;  
  
procedure VectorTest is  
    V : Vector.Vector;  
begin  
    Vector.Set(V, X => 3.0, Y => 4.0, Z => 5.0);  
    Vector.Print(V);  
    Vector.Normalise(V);  
    Vector.Print(V);  
end VectorTest;
```

Because the compiler needs to know how big a **Vector** is when it compiles client code that uses **Vectors**

Enumerations



```
procedure Enum is
  type Day is (Monday, Tuesday, Wednesday, Thursday, Friday,
               Saturday, Sunday);

  Temperatures : array (Day) of Float;

  Mo : Day;
  Su : Day;
begin
  Mo := Day'First;
  Su := Day'Last;

  if Mo < Su then
    Ada.Text_IO.Put("Monday < Sunday");
  end if;

  for D in Day'Range loop
    Temperatures(D) := 27.0;
  end loop;
end Enum;
```

attributes

comparison

array
indexed by
enums

A Note on Standard Types



THE UNIVERSITY OF
MELBOURNE

Which is best?

This one:

```
type SecondOfDay is range 0 .. 86_400;
```

More portable than this:

```
type SecondOfDay is new Integer range 0 .. 86_400;
```

More static checking than this:

```
subtype SecondOfDay is Integer range 0 .. 86_400;
```

Resources



Ada 2012 Reference Manual:

http://www.ada-auth.org/standards/rm12_w_tc1/html/RM-TTL.html

(including what's in the standard library,
plus all the language features we don't cover:
dynamic memory, variant records,
OO, concurrency, etc.)

Ada Quality and Style Guide:

https://en.wikibooks.org/wiki/Ada_Style_Guide